

Manual de usuario

Analizador semántico para Pascal

Asignatura: Lenguajes y Autómatas II

Versión: 1.0.0

Descarga de código fuente: Para ejecutar este proyecto en su propio servidor local o inspeccionar el código, puede descargar el proyecto completo desde nuestro repositorio oficial en GitHub:

<https://github.com/20pedro01/Analizador-sem-ntico-de-Pascal>

1. Introducción

Este software es una herramienta diseñada para verificar la corrección semántica de programas escritos en lenguaje Pascal. Su objetivo principal es ir más allá de la revisión sintáctica básica, asegurando que el código tenga sentido lógico y cumpla con las reglas de tipado estricto del lenguaje.

El sistema valida de forma exhaustiva:

- **Compatibilidad de tipos de datos estricta:** Evita asignaciones inválidas, controla operaciones aritméticas, relacionales y lógicas, prohibiendo el *narrowing* automático (ejemplo de Real a Integer).
- **Declaración e inicialización:** Verifica que las variables existan antes de usarse y advierte si se leen sin valor inicial.
- **Alcance (scope) y ambigüedad:** Soporta múltiples niveles de anidamiento, detectando colisiones de nombres (shadowing) y ambigüedades de tipo entre scopes.
- **Control de ciclo FOR:** Protege la variable iteradora contra modificaciones internas y exige que sea de tipo ordinal.
- **Subprogramas (funciones y procedimientos):** Exige la asignación obligatoria de valores de retorno, prohíbe ignorar llamadas a funciones y valida estrictamente la cantidad y los tipos de parámetros enviados.
- **Arreglos (arrays):** Evalúa matemáticamente las expresiones de los índices, exigiendo siempre resultados enteros.

2. Tipos de datos admitidos

El simulador soporta los tipos de datos estándar de Pascal necesarios para la enseñanza de lógica de programación y estructuras de datos:

Tipo	Descripción	Ejemplo
<code>integer</code>	Números enteros de 32 bits. Utilizado para contadores e índices.	<code>42</code> , <code>-10</code>
<code>real</code>	Números con punto decimal (punto flotante).	<code>3.14</code> , <code>0.001</code>
<code>boolean</code>	Valores lógicos.	<code>true</code> , <code>false</code>
<code>char</code>	Un solo carácter alfanumérico o símbolo.	<code>'A'</code> , <code>'#'</code>
<code>string</code>	Cadenas de texto.	<code>'Hola Mundo'</code>
<code>array</code>	Colecciones indexadas de datos del mismo tipo.	<code>array[1..10] of integer</code>

Nota sobre compatibilidad: El sistema permite la asignación automática de `integer` a `real` , pero marcará un **error** si intenta asignar un `real` a un `integer` sin redondear, para evitar pérdida de precisión no intencionada.

3. Requisitos del sistema

- **Sistema operativo:** Windows, Linux o macOS.
- **Lenguaje base:** PHP 8.0 o superior.
- **Navegador:** Chrome, Firefox, Safari o Edge.

4. Guía de uso (interfaz web)

Paso 1: Abrir la aplicación

1. Navegue a la carpeta del proyecto.
2. Ejecute el servidor local con el comando: `php -S localhost:XXXX` (sustituya las XXXX por la dirección real de su servidor, ejemplo 8080).
3. Abra su navegador en `http://localhost:XXXX` (sustituya las XXXX por la dirección real de su servidor).

Paso 2: Cargar o escribir código

En el editor principal, puede escribir su código Pascal. Para facilitar las pruebas, utilice la barra lateral:

- **Botones verdes:** Cargan ejemplos que compilan correctamente.

- **Botones amarillos:** Cargan ejemplos con advertencias (warnings) que permiten la ejecución pero sugieren mejoras.
- **Botones rojos:** Cargan ejemplos con errores intencionales para probar el validador.

También puede añadir archivos .pas desde su computadora.

Paso 3: Analizar

Haga clic en el botón "**Analizar código**". El sistema procesará el texto lexicamente, sintácticamente y semánticamente.

5. Interpretación de resultados

El simulador clasifica los mensajes encontrados durante el análisis en dos categorías principales:

 Errores

Son violaciones a las reglas del lenguaje que impiden que el programa sea lógico o ejecutable. Deben corregirse obligatoriamente.

- Uso de variables no declaradas.
- Incompatibilidad de tipos.
- Redefinición de identificadores.
- Modificar la variable de un ciclo FOR.

 Advertencias (warnings)

Indican posibles malas prácticas o situaciones sospechosas que no rompen el programa pero podrían causar bugs.

- Variables declaradas que nunca se usan.
- Uso de una variable no inicializada.
- Ignorar el valor de retorno de una función.

Escenario A: Compilación exitosa

Si el código es correcto, verá un mensaje verde de éxito y la **Tabla de símbolos** generada.

Nombre	Tipo	Scope	Categoría
x	Integer	Global	Variable

Nombre	Tipo	Scope	Categoría
Pi	Real	Global	Constante

Escenario B: Errores semánticos

Si hay errores, se mostrará una lista detallada indicando la línea y la causa.

Ejemplo de advertencia:

[Línea 2] Advertencia: La variable 'z' ha sido declarada pero nunca se utiliza en el código.

Caso 1: Error de tipado estructural (narrowing y operadores)

```
VAR iterador: INTEGER;
    flag: BOOLEAN;
BEGIN
    iterador := 3.1416; { ✖ Error: Asignando Real a Entero }
    flag := iterador + 5; { ✖ Error: Operación aritmética no compatible con Boolean }
END.
```

Explicación: Pascal es fuertemente tipado. No permite asignar decimales a enteros sin conversión explícita (Trunc/Round) ni mezclar aritméticas en tipos lógicos.

Caso 2: Error de ambigüedad (shadowing)

```
VAR varLocal: INTEGER;

PROCEDURE Test;
VAR varLocal: BOOLEAN;
BEGIN
    varLocal := TRUE;
END;

BEGIN
    varLocal := 10;
END.
```

Explicación: Aunque es sintácticamente válido redefinir una variable en un scope local, el analizador lo marca como un error crítico de **Ambigüedad de tipo** porque `varLocal` es entero a nivel global pero booleano a nivel local, siendo una alta fuente de bugs.

Caso 3: Verificación estricta de subprogramas

```
PROCEDURE MiProc(x: INTEGER; y: BOOLEAN);
BEGIN
    x := x + 1;
END;

FUNCTION CalculaVentas: INTEGER;
BEGIN
    x := 10;
    { ✖ Error: Función termina sin devolver valor }
END;

BEGIN
    MiProc(10, 20); { ✖ Error: El parámetro 2 esperaba BOOLEAN, se obtuvo INTEGER }
    CalculaVentas; { ⚠ Warning: Valor de retorno ignorado }
END.
```

Explicación: El analizador verifica individualmente cada argumento enviado a un subprograma. Además, asegura que las funciones siempre asignen su valor de retorno a sí mismas y que quien las llama realmente utilice dicho resultado.

Caso 4: Acceso a arreglos

```
VAR Arr: ARRAY[1..10] OF INTEGER;
    Indice: STRING;
BEGIN
    Indice := 'Cinco';
    Arr[Indice] := 100; { ✖ Error: El índice debe ser INTEGER }
END.
```

Explicación: No basta con que una variable sea un arreglo; cualquier expresión utilizada entre corchetes `[]` será evaluada para garantizar que matemáticamente resulte en un número entero.

Caso 5: Advertencias por código no óptimo

```
VAR x, y, z: INTEGER;
    nombre: STRING;
BEGIN
    x := 10;
    y := 20;
```

```
{ ⚠ Warning: 'z' declarada pero nunca usada }
{ ⚠ Warning: 'nombre' declarada pero nunca usada }
writeln(x + y);
END.
```

Explicación: A diferencia de los errores, las advertencias no bloquean la ejecución. El analizador detecta variables que ocupan memoria pero no tienen propósito, ayudando a limpiar y optimizar el código fuente. También advertirá si se intenta leer una variable que no ha recibido un valor previo (no inicializada).

6. Solución de problemas

P: ¿El analizador se queda en blanco?

R: Asegúrese de que su código PHP tenga permisos de ejecución y que no haya errores de sintaxis en el archivo `index.php`.

P: ¿Por qué marca error en la última línea?

R: Verifique que el programa termina con `END.` (con punto final).